

DTS103TC

Data Structure and Algorithms

Lab 9: Minimum Spanning Tree and Shortest Path

Dr. Qi Chen, Dr. Di Zhang, Dr. Md Maruf Hasan,
Dr. Xuechen Liu, Dr. Guangyu Ren and Dr. Bin Chen
School of AI and Advanced Computing

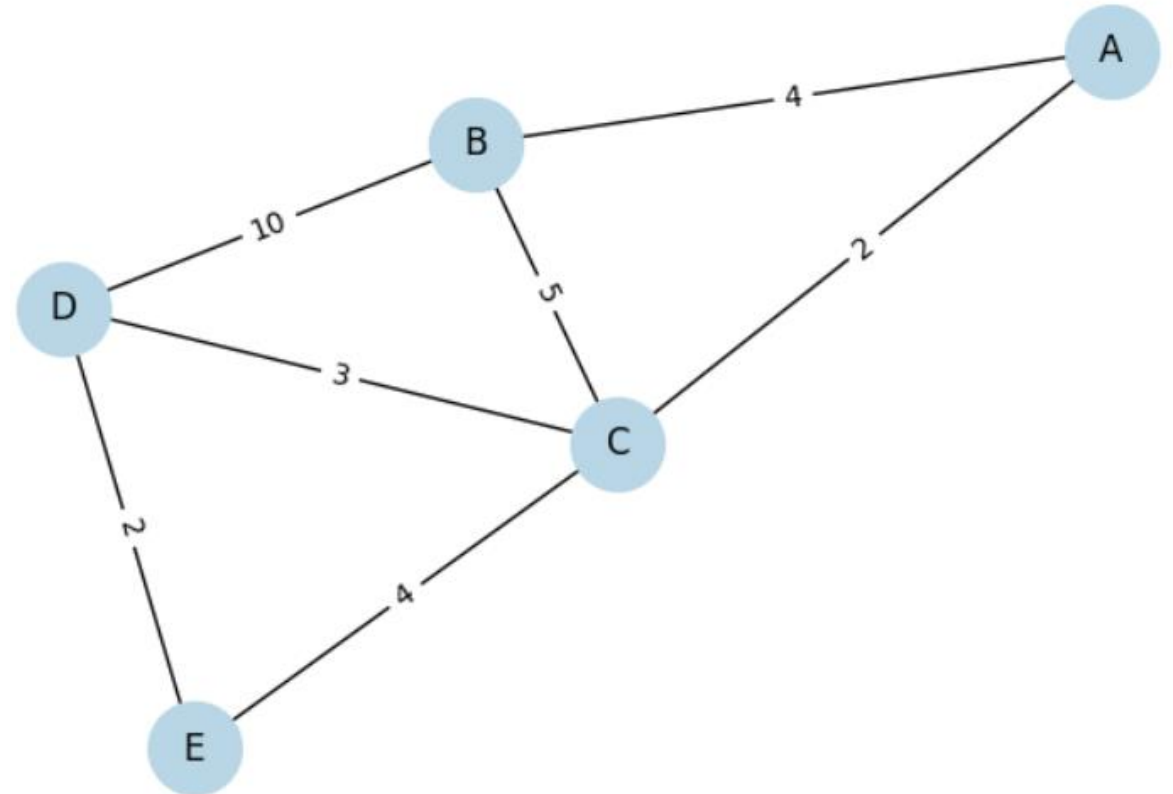
Learning outcome

- Graph Visualization with NetworkX
- Graph Algorithms Implementation
 - Kruskal's algorithm for minimum spanning tree
 - Dijkstra's algorithm for shortest path

Graph Visualization

- Graphs are often stored in code (e.g., adjacency list)
- But code is hard to read for humans
- Visualization helps us:
 - understand the structure
 - see connections clearly
 - verify algorithm results

```
graph = {  
  'A': [('B', 4), ('C', 2)],  
  'B': [('A', 4), ('C', 5), ('D', 10)],  
  'C': [('A', 2), ('B', 5), ('D', 3), ('E', 4)],  
  'D': [('B', 10), ('C', 3), ('E', 2)],  
  'E': [('C', 4), ('D', 2)]  
}
```



Graph Visualization with NetworkX

- What is NetworkX?
 - A Python library for working with graphs
 - Allows us to:
 - create graphs
 - add nodes and edges
 - visualize graphs
- We will use NetworkX mainly as a visualization tool in this lab.

Example: Weighted Graph

- We can represent the graph using an edge list.
 - Each tuple represents one edge
 - Example: (A, B, 4) means
 - there is an edge between A and B
 - with weight 4
- This is an undirected graph, so (A, B) is the same as (B, A).
- The weight can represent cost, distance, or time.

```
import networkx as nx
G = nx.Graph()

edges = [
    ('A', 'B', 4),
    ('A', 'C', 2),
    ('B', 'C', 5),
    ('B', 'D', 10),
    ('C', 'D', 3),
    ('C', 'E', 4),
    ('D', 'E', 2)
]

G.add_weighted_edges_from(edges)
```

Graph Visualization (NetworkX Output)

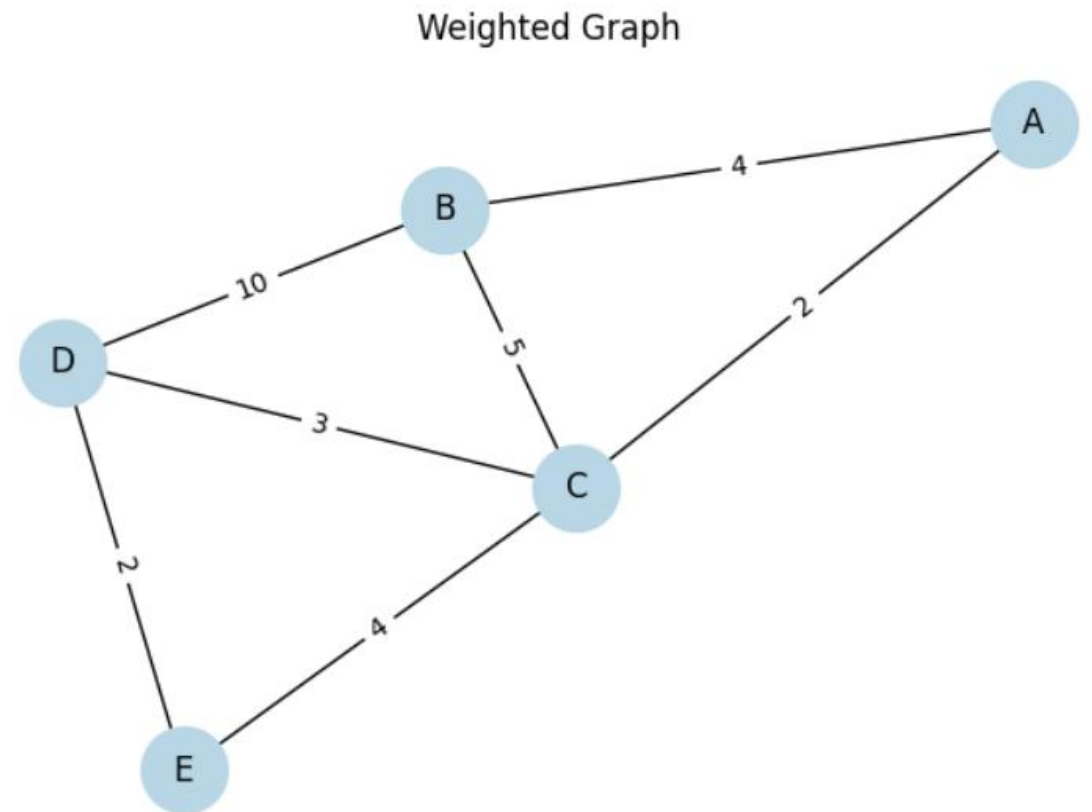
```
plt.figure(figsize=(6, 4))

pos = nx.spring_layout(G, seed=42)

nx.draw(
    G,
    pos,
    with_labels=True,
    node_color='lightblue',
    node_size=1000,
    font_size=12
)

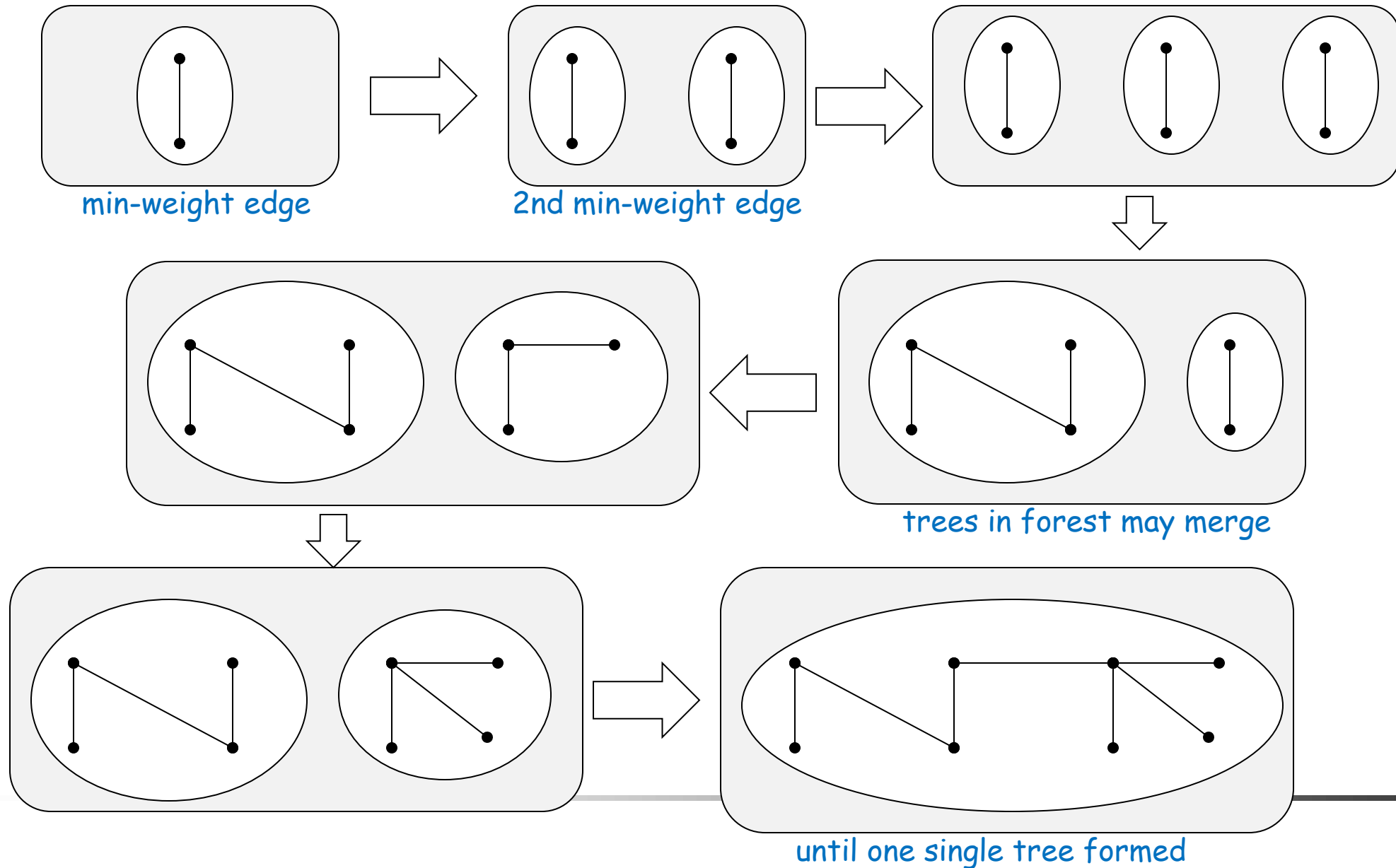
edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)

plt.title("Weighted Graph")
plt.axis('off')
plt.show()
```

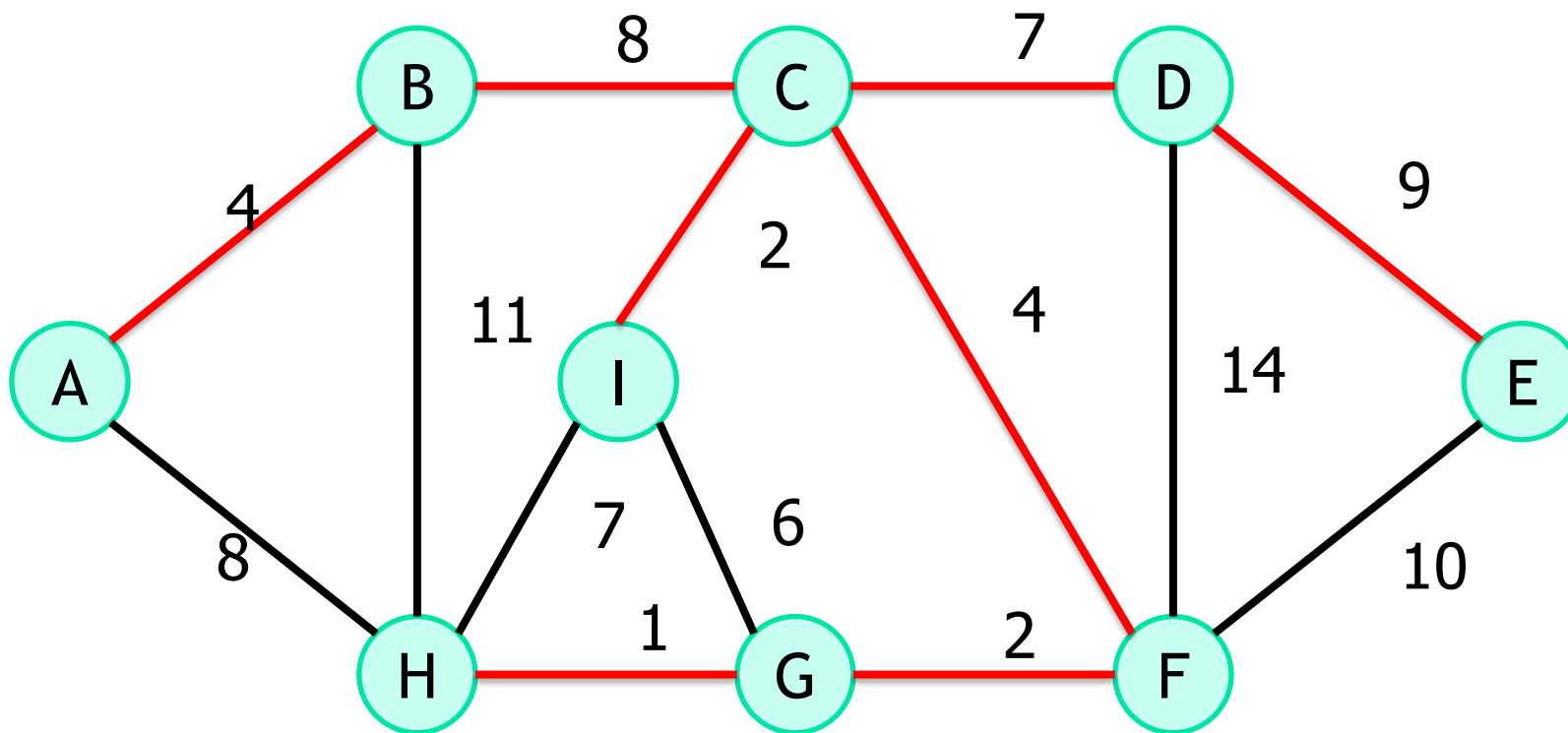


Kruskal's algorithm

Idea of Kruskal's algorithm - MST



Kruskal's algorithm - MST



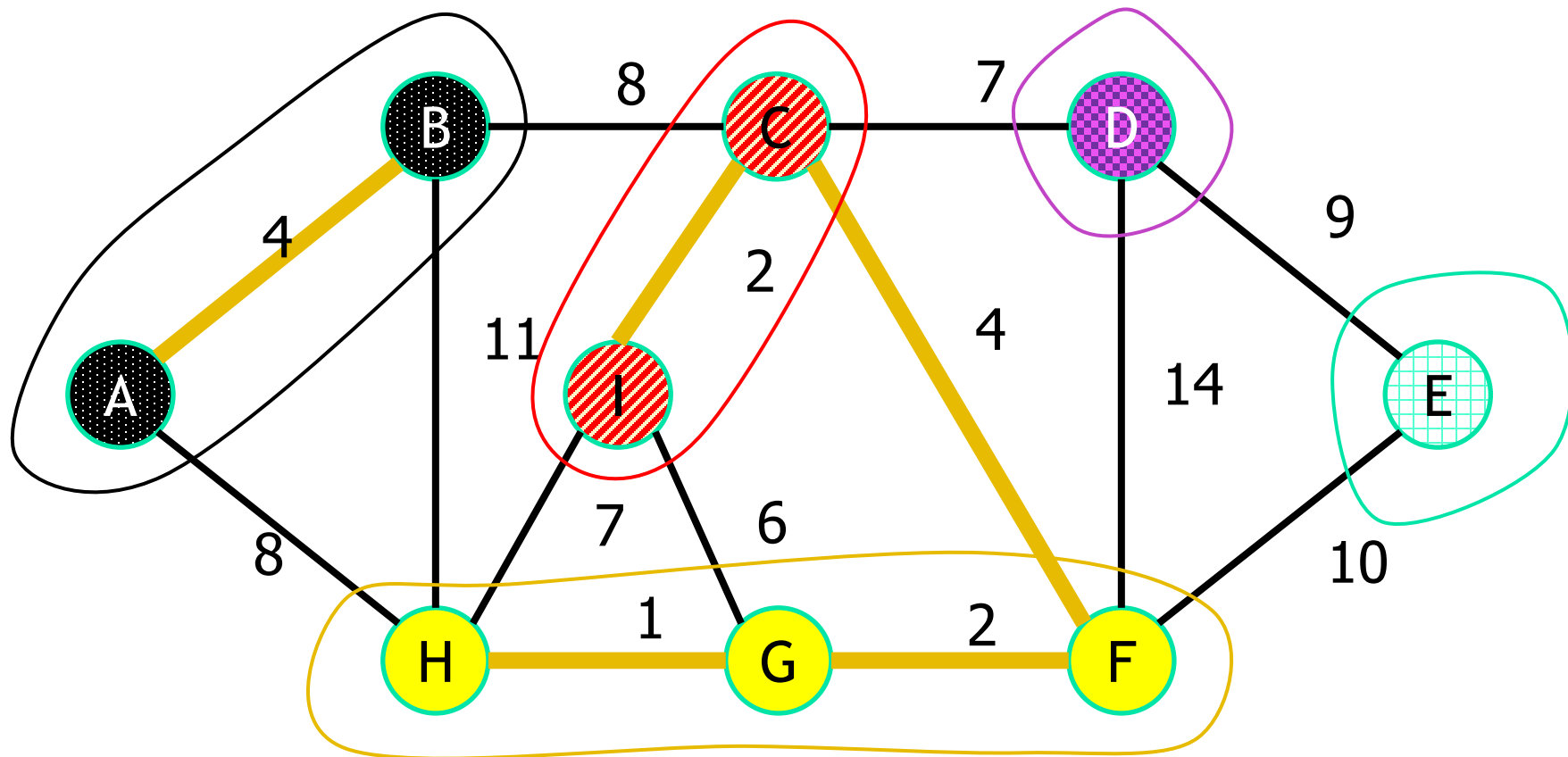
<i>(h,g)</i>	<i>1</i>
<i>(i,c)</i>	<i>2</i>
<i>(g,f)</i>	<i>2</i>
<i>(a,b)</i>	<i>4</i>
<i>(c,f)</i>	<i>4</i>
(i,g)	6
<i>(c,d)</i>	<i>7</i>
(h,i)	7
<i>(b,c)</i>	<i>8</i>
(a,h)	8
<i>(d,e)</i>	<i>9</i>
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm

When we add an edge, we merge two trees:

A **forest** is a collection of disjoint trees

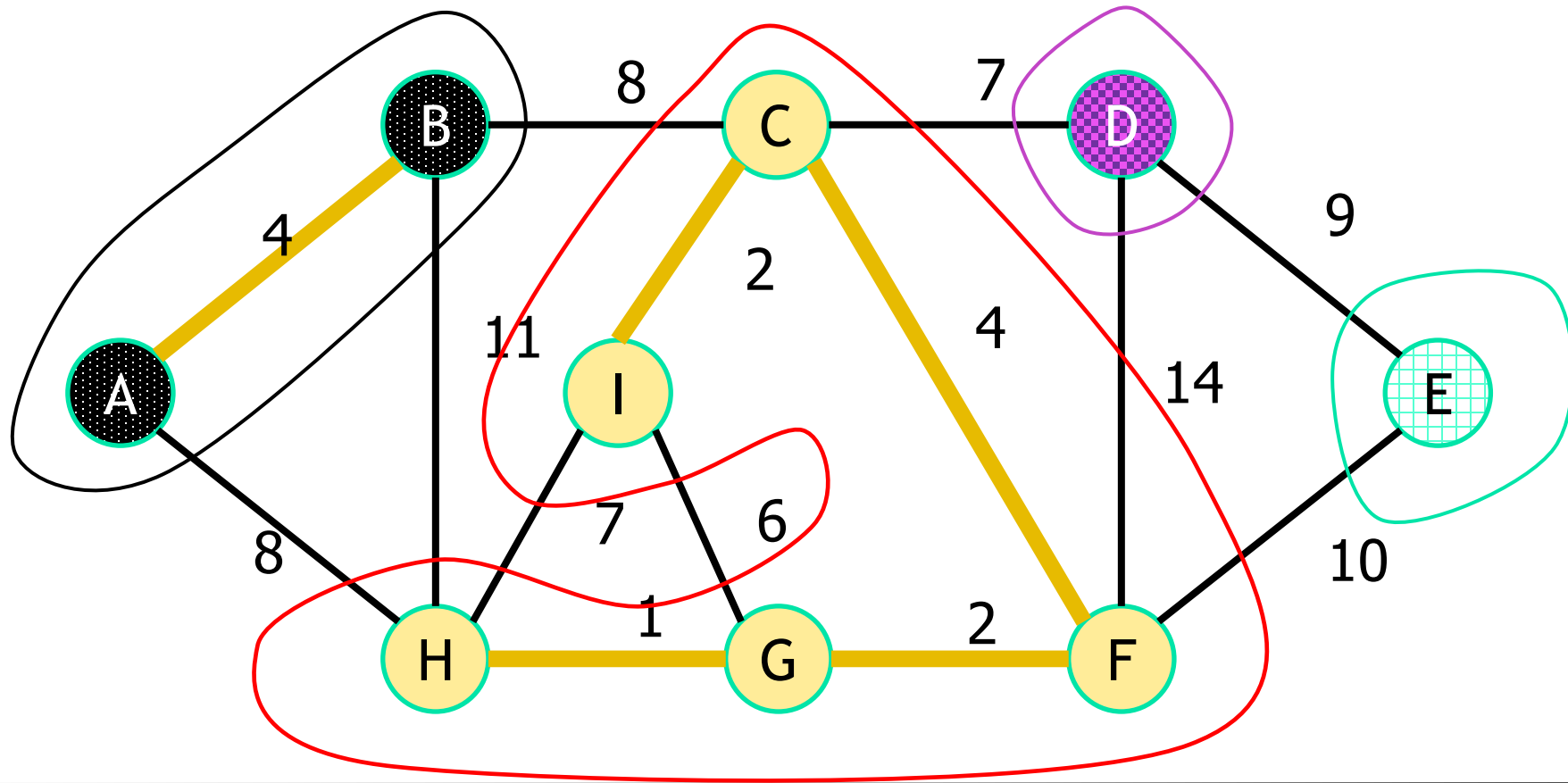


Kruskal's algorithm

When we add an edge, we merge two trees.

We never add an edge within a tree since that would create a cycle

A **forest** is a collection of disjoint trees



Disjoint-set data structure

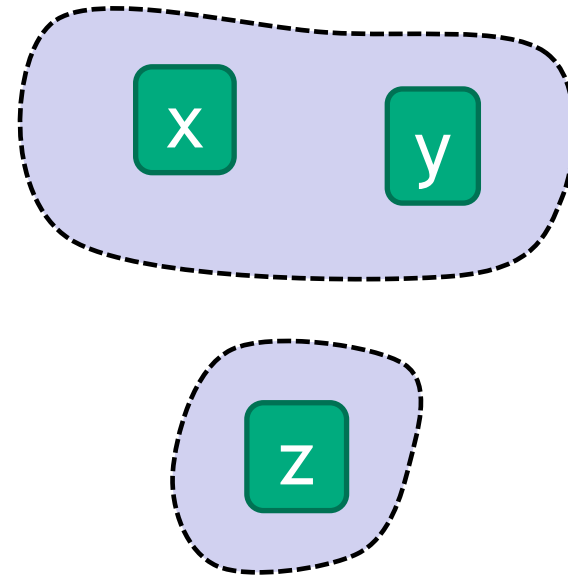
- Used for storing collections of sets
- Supports:
 - **makeSet(u)**: create a set {u}
 - **find(u)**: return the set that u is in
 - **union(u,v)**: merge the set that u is in with the set that v is in.

`makeSet (x)`

`makeSet (y)`

`makeSet (z)`

`union (x, y)`



Disjoint-set data structure

- Used for storing collections of sets
- Supports:
 - **makeSet(u)**: create a set {u}
 - **find(u)**: return the set that u is in
 - **union(u,v)**: merge the set that u is in with the set that v is in.

```
class DisjointSet:
    def __init__(self):
        self.parent = {}
        self.rank = {}

    def make_set(self, u):
        self.parent[u] = u
        self.rank[u] = 0

    def find(self, u):
        if self.parent[u] != u:
            self.parent[u] = self.find(self.parent[u]) # path compression
        return self.parent[u]

    def union(self, u, v):
        root_u = self.find(u)
        root_v = self.find(v)

        if root_u == root_v:
            return False

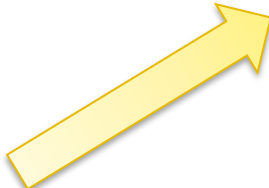
        # union by rank
        if self.rank[root_u] < self.rank[root_v]:
            self.parent[root_u] = root_v
        elif self.rank[root_u] > self.rank[root_v]:
            self.parent[root_v] = root_u
        else:
            self.parent[root_v] = root_u
            self.rank[root_u] += 1
        return True
```

Kruskal's algorithm implementation

Kruskal's algorithm is a **greedy algorithm**.

It works as follows:

- Sort all edges by weight in increasing order
- Add the smallest edge if it does not form a cycle
- Repeat until we have enough edges for the MST

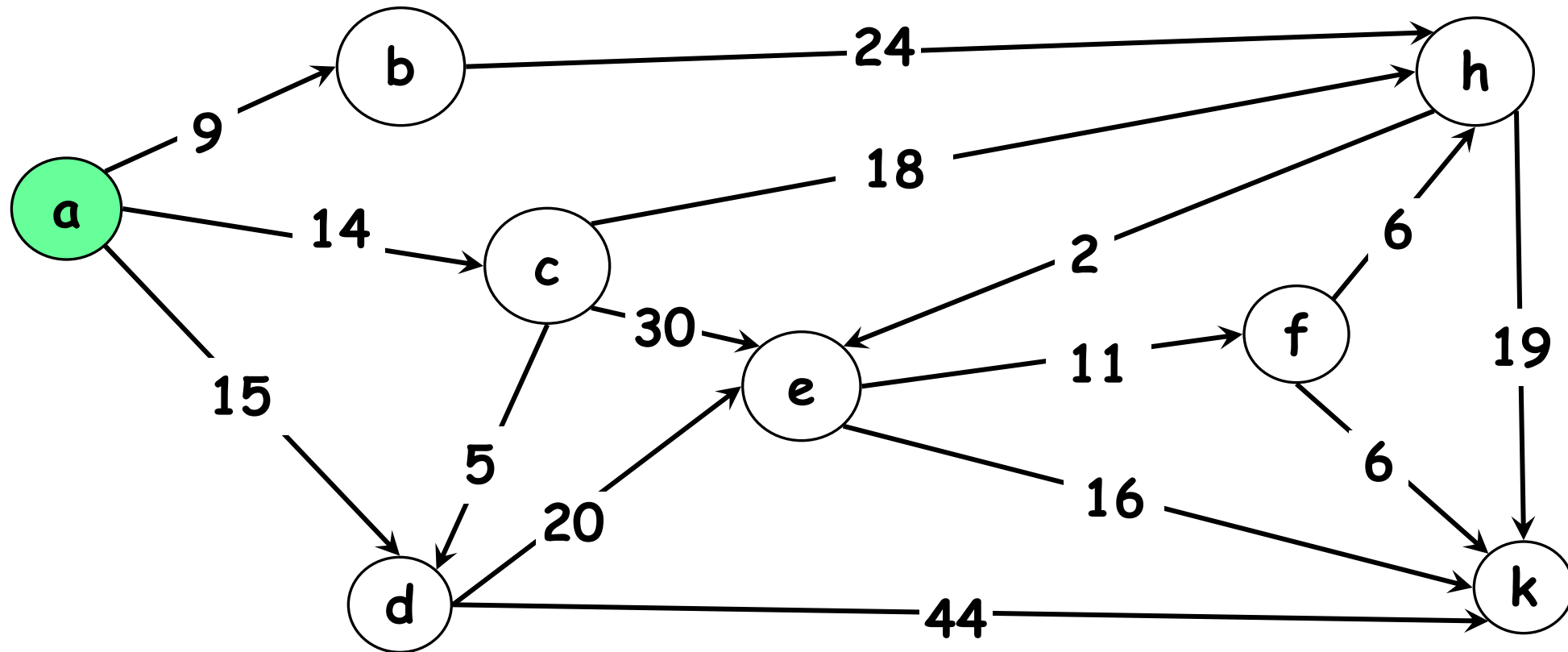


```
def kruskal(nodes, edges):  
    #sort edges by weight  
    sorted_edges = sorted(edges, key=lambda x: x[2])  
  
    #initialize disjoint set  
    ds = DisjointSet()  
    for u in nodes:  
        ds.make_set(u)  
  
    mst = []  
    total_weight = 0  
  
    #process edges in increasing order  
    for u, v, w in sorted_edges:  
        if ds.find(u) != ds.find(v):  
            ds.union(u, v)  
            mst.append((u, v, w))  
            total_weight += w  
  
    return mst, total_weight
```

Dijkstra algorithm

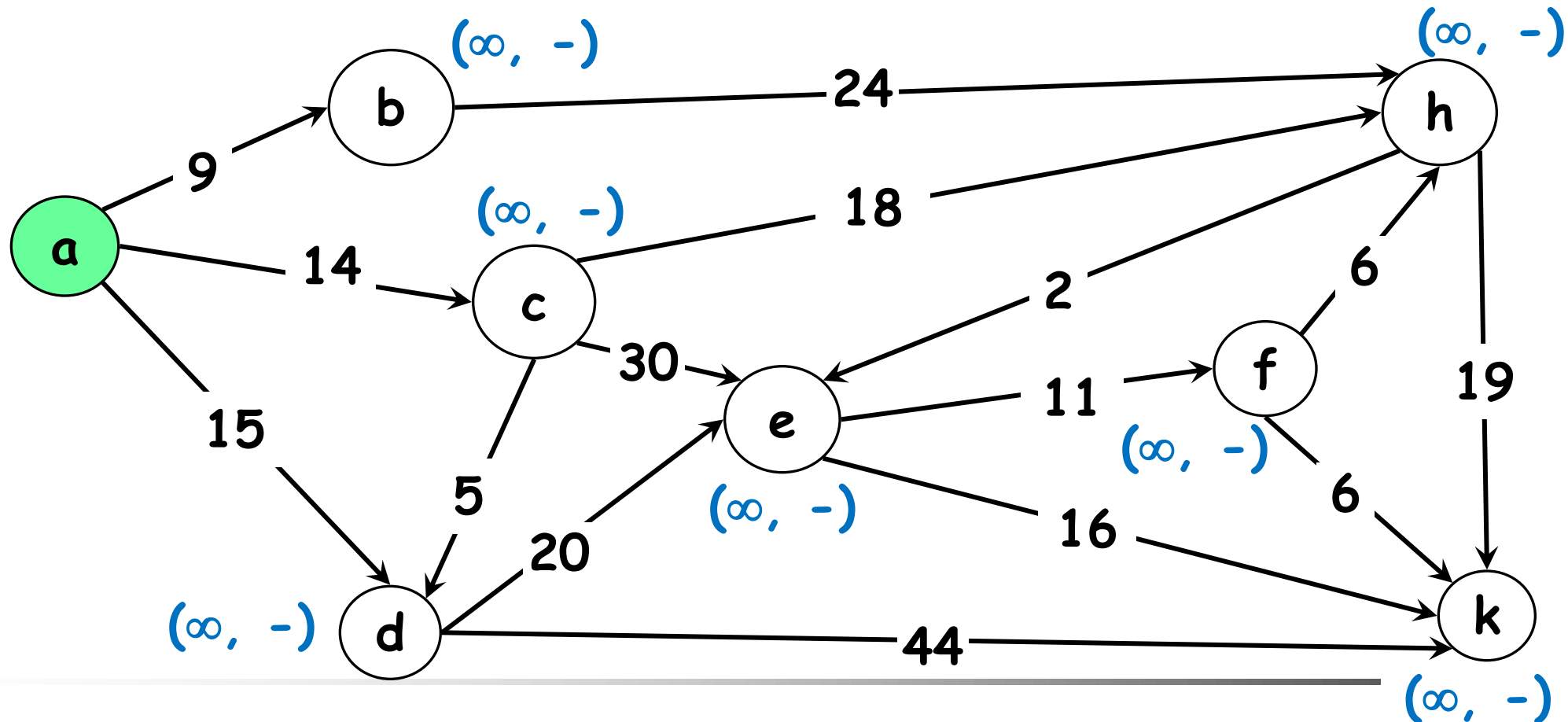
Dijkstra's algorithm

Suppose vertex **a** is the source, we now show how Dijkstra's algorithm works



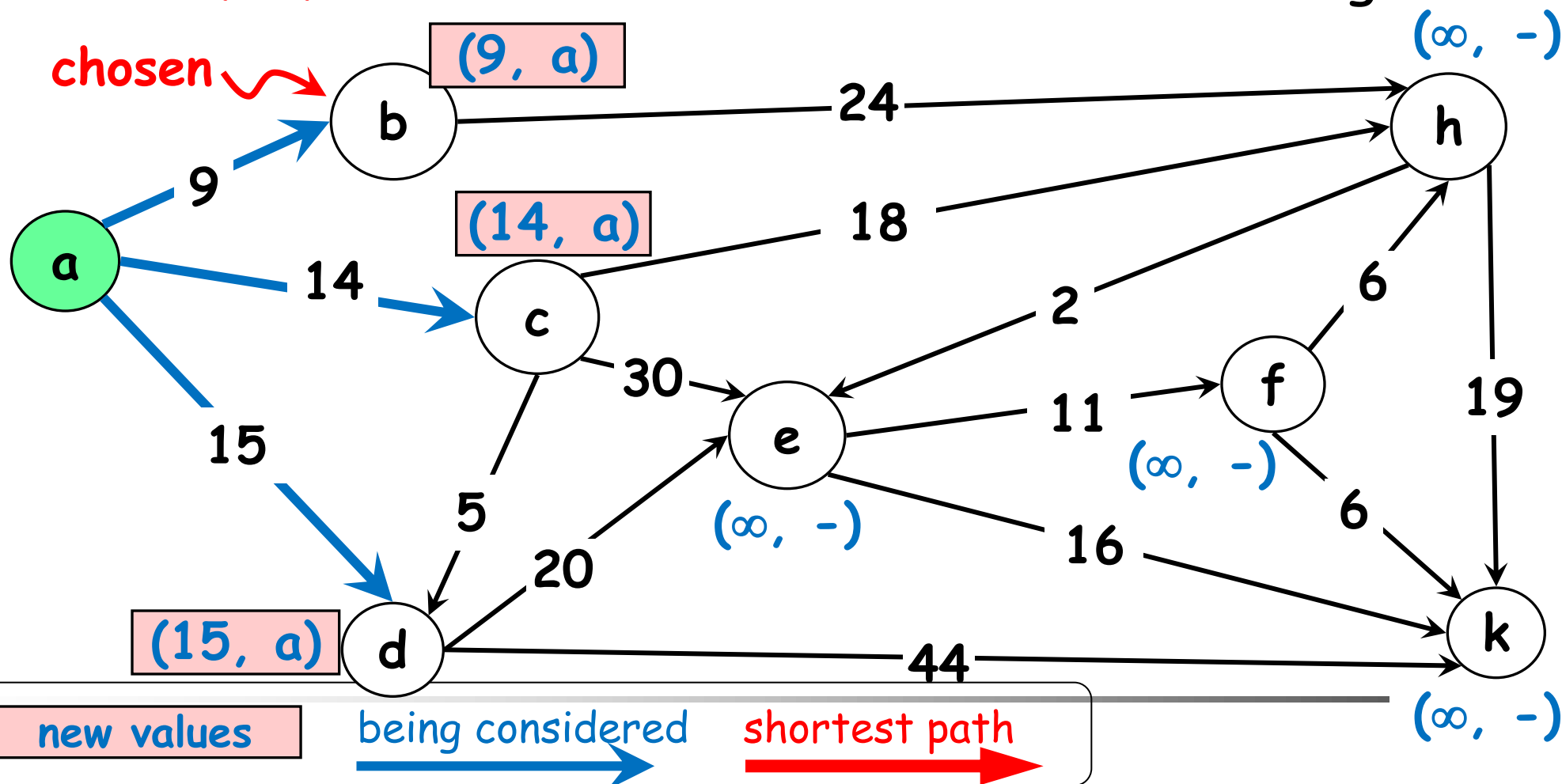
Dijkstra's algorithm

Every vertex v keeps 2 labels: (1) the weight of the current shortest path from a ; (2) the vertex leading to v on that path, initially as $(\infty, -)$



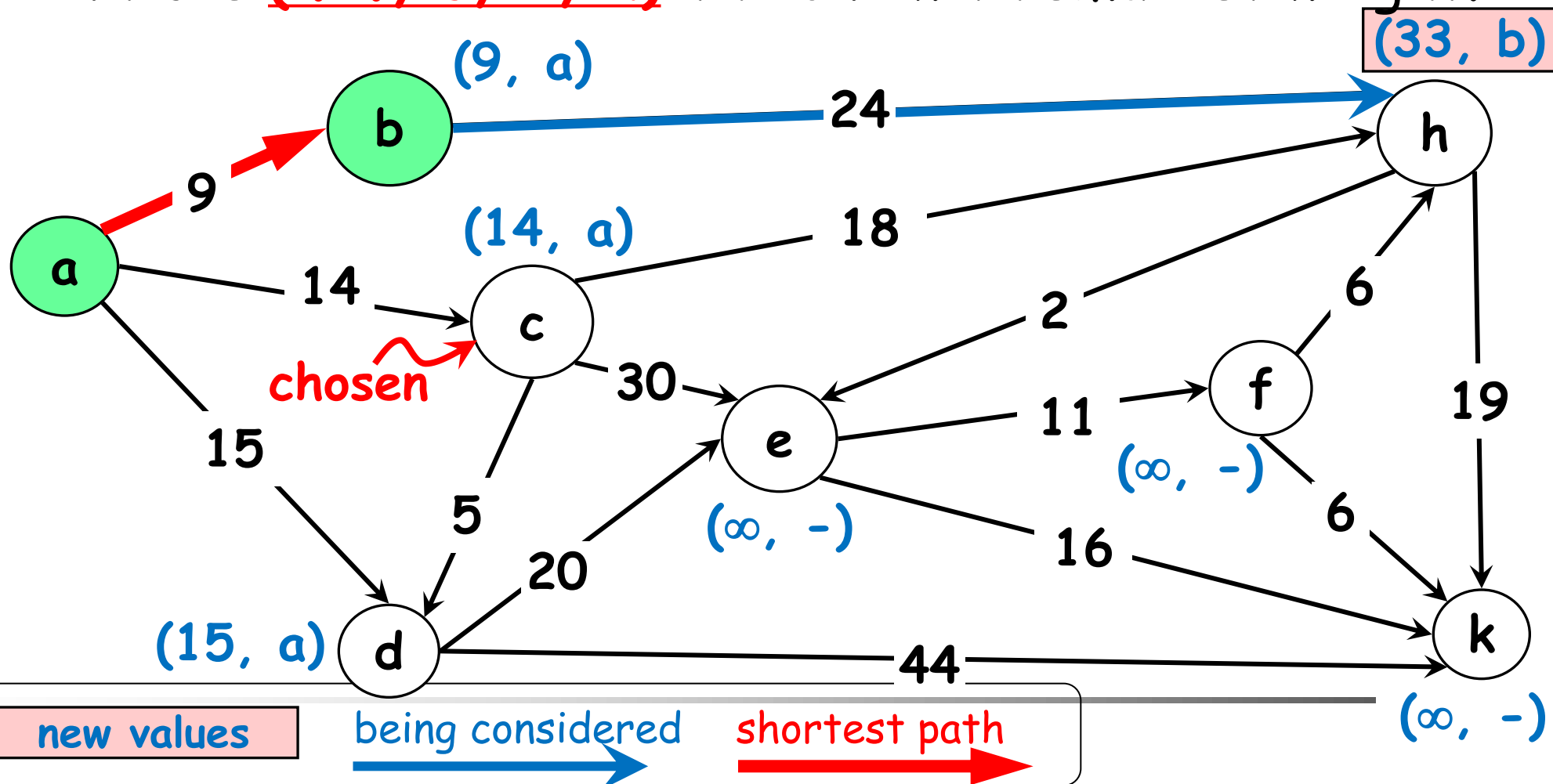
Dijkstra's algorithm

For every neighbor u of a , update the weight to the weight of (a, u) and the leading vertex to a . Choose from b, c, d the one with the smallest such weight.



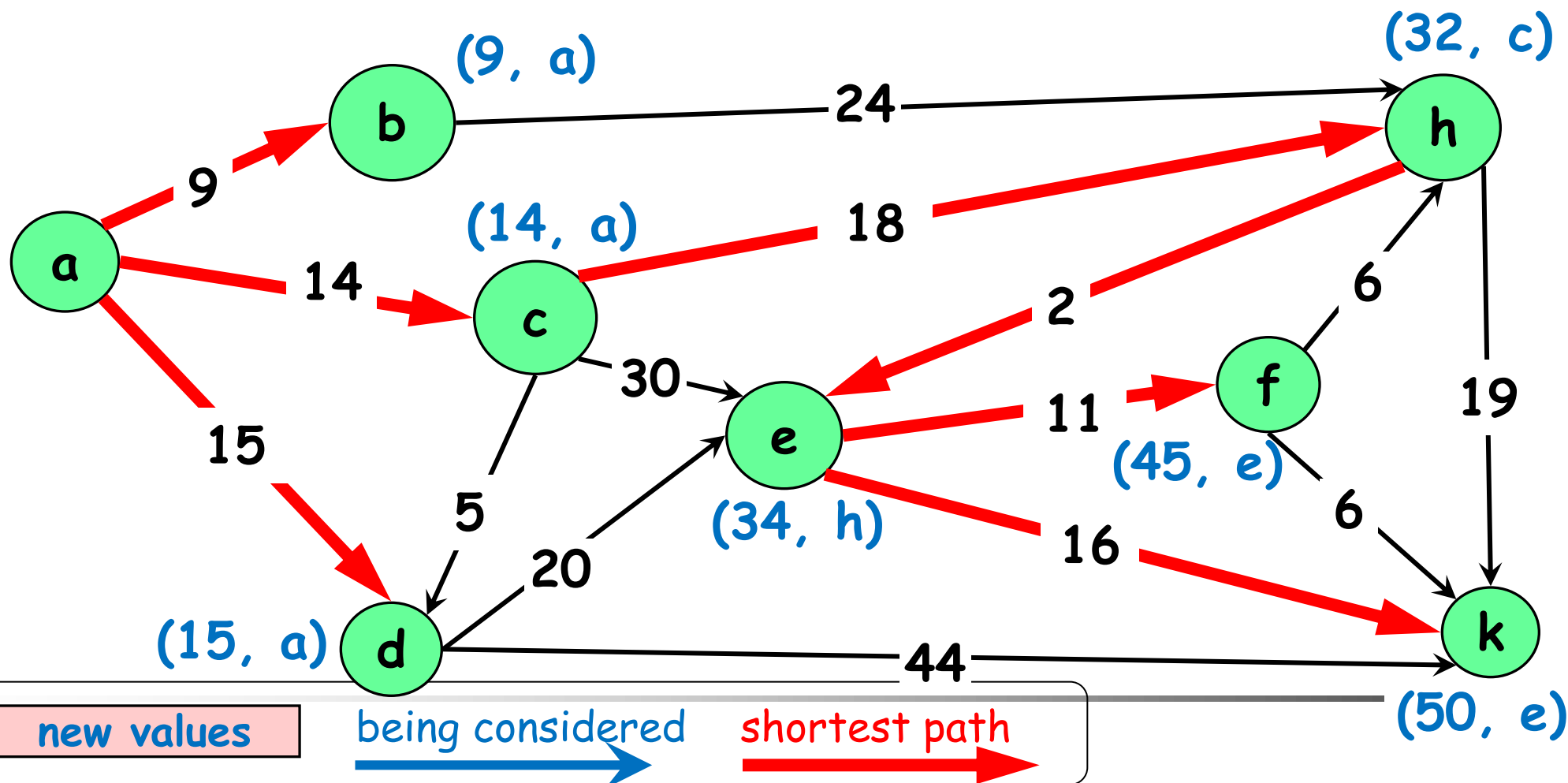
Dijkstra's algorithm

For every un-chosen neighbor of vertex **b**, update the weight and leading vertex. Choose from **ALL** un-chosen vertices (*i.e.*, **c, d, h**) the one with smallest weight.



Dijkstra's algorithm

At this point, all vertices are chosen, and the shortest path from **a** to every vertex is discovered.



Dijkstra's algorithm implementation

Dijkstra's algorithm is a **greedy algorithm**.

It works as follows:

- Set the distance to the source as 0; Set all other distances as infinity
- Repeatedly choose the vertex with the smallest current distance
- Update the distances of its neighbors

```
import heapq
def dijkstra(graph, source):
    dist = {node: float('inf') for node in graph}
    prev = {node: None for node in graph}
    dist[source] = 0

    pq = []
    heapq.heappush(pq, (0, source))

    while pq:
        current_dist, u = heapq.heappop(pq)

        if current_dist > dist[u]:
            continue

        for v, weight in graph[u]:
            new_dist = current_dist + weight

            if new_dist < dist[v]:
                dist[v] = new_dist
                prev[v] = u
                heapq.heappush(pq, (new_dist, v))

    return dist, prev
```

Please work on Coursework Task 4 after you have finished the lab content.